

# LookupViT: Compressing visual information to a limited number of tokens

Rajat Koner<sup>1,2\*</sup>, Gagan Jain<sup>1\*</sup>,  
Prateek Jain<sup>1</sup>, Volker Tresp<sup>2</sup>, and Sujoy Paul<sup>1</sup>

<sup>1</sup> Google DeepMind

<sup>2</sup> Ludwig Maximilian University of Munich

**Abstract.** Vision Transformers (ViT) have emerged as the de-facto choice for numerous industry grade vision solutions. But their inference cost can be prohibitive for many settings, as they compute self-attention in each layer which suffers from quadratic computational complexity in the number of tokens. On the other hand, spatial information in images and spatio-temporal information in videos is usually sparse and redundant. In this work, we introduce LookupViT, that aims to exploit this information sparsity to reduce ViT inference cost. LookupViT provides a novel general purpose vision transformer block that operates by compressing information from higher resolution tokens to a fixed number of tokens. These few compressed tokens undergo meticulous processing, while the higher-resolution tokens are passed through computationally cheaper layers. Information sharing between these two token sets is enabled through a bidirectional cross-attention mechanism. The approach offers multiple advantages - (a) easy to implement on standard ML accelerators (GPUs/TPUs) via standard high-level operators, (b) applicable to standard ViT and its variants, thus generalizes to various tasks, (c) can handle different tokenization and attention approaches. LookupViT also offers flexibility for the compressed tokens, enabling performance-computation trade-offs in a single trained model. We show LookupViT’s effectiveness on multiple domains - (a) for image-classification (ImageNet-1K and ImageNet-21K), (b) video classification (Kinetics400 and Something-Something V2), (c) image captioning (COCO-Captions) with a frozen encoder. LookupViT provides 2× reduction in FLOPs while upholding or improving accuracy across these domains. In addition, LookupViT also demonstrates out-of-the-box robustness and generalization on image classification (ImageNet-C,R,A,O), improving by up to 4% over ViT.

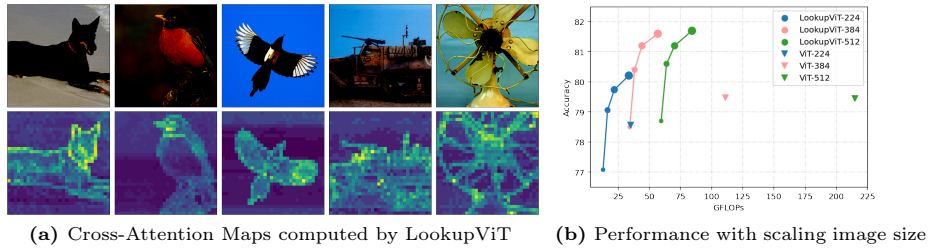
**Keywords:** token compression · multi-resolution · elastic inference

## 1 Introduction

Images and videos, the cornerstones of modern visual communication, possess an inherent characteristic: their information content is often sparse and exhibits

---

\* denotes equal contribution



**Fig. 1:** (a) Cross-attention maps between compressed and lookup tokens, emphasizing LookupViT’s ability to extract relevant information from lookup tokens as needed for classification. (b) LookupViT vs ViT while scaling image resolution. The individual points per curve are for varied compressed tokens sizes ( $3 \times 3$ ,  $5 \times 5$ ,  $7 \times 7$ ,  $10 \times 10$ ). LookupViT scales quite efficiently w.r.t ViT.

significant redundancy. However, Vision Transformers (ViTs) [12], despite their dominance across multiple vision tasks, do not exploit this redundancy and attend to every token in a homogenized way. This leads to quadratic computational complexity with respect to image size, hindering its applicability in real-time situations. To bridge this gap, there is a pressing need to efficiently compress visual information into a smaller, more computationally manageable set of tokens. Such representations would unlock the potential of ViTs for resource-constrained scenarios while preserving their flexibility and performance advantages which led to their widespread adoption in the field of computer vision.

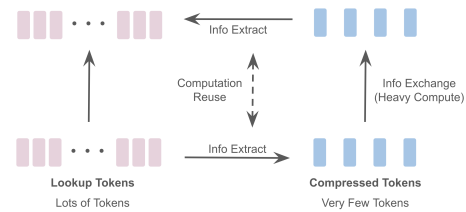
Several architectures aim to address the computational burden of ViTs by thoughtfully reducing the number of tokens. Token pruning methods retain a subset of tokens [14, 26, 31], while token pooling techniques combine similar tokens for a more compact representation [5, 23]. These mechanisms rely on heuristics derived from attention scores or feature similarities and might require additional task-specific adjustments. While these techniques offer valuable benefits, they may necessitate further fine-tuning based on the application. In contrast, we propose a novel LookupViT block to replace the vanilla ViT block, which intrinsically acts as a compression module. This design eliminates the need for post-processing or extensive fine-tuning. Furthermore, our method preserves the general structure of the ViT architecture, thus allowing further optimization and adaptations using existing approaches like token pruning or merging.

Compression modules like TokenLearner [25] and Perceiver [19] have also been explored in the literature. TokenLearner utilizes vanilla ViT blocks for a significant portion of the network depth, compressing a large number of tokens to a smaller set (e.g., 8 or 16) at later stages. This reliance on ViT blocks incurs a substantial computation and heavily limits the full utilization of compression module within the network. Perceiver, on the other hand, devises an asymmetric information flow directly from image pixels to a small set of latent representations iteratively throughout the network. Moreover, for these network architectures, it is non-trivial to extract multiple models with the same

parameters, to exhibit a compute-performance trade-off between extracted models. LookupViT distinguishes itself by offering a scalable, computationally efficient block that can be seamlessly repeated like standard ViT blocks. Its bidirectional cross-attention mechanism facilitates a richer exchange of information between the compressed and original tokens, enhancing representational power.

In this paper, we corroborate that for innately redundant modalities like vision, condensing relevant spatial (and temporal) information from original tokens to a much smaller set can still sustain performance while significantly lowering the computational requirements, by maintaining an effective exchange of information between the two token sets. Figure 1b indicates LookupViT’s ability to scale to large image sizes efficiently, by processing only relevant information, compared to vanilla ViT blocks, which scales quadratically in the number of original image tokens. We denote the smaller compressed set of tokens as *compressed* tokens, which "look" at the larger original set of tokens, which we call *lookup* tokens. The information exchange between these tokens happens in every LookupViT block in three key steps, as shown in Figure 2 - (i) cross attention to transfer relevant information from the lookup tokens to the compressed tokens (shown in Figure 1a), (ii) self-attention amongst the compressed tokens, and (iii) information transfer from the compressed tokens to the lookup tokens using shared attention weights, computed in the first step. While the compressed tokens communicate through self-attention, the lookup tokens communicate among themselves only via the compressed tokens. This technique avoids the quadratic scaling, while ensuring that the lookup latent representations get richer along the layers.

LookupViT’s intrinsic design naturally supports flexibility in terms of token compression and variable image or token size. By adjusting the down-sampling ratio between compressed and lookup tokens, the cost-performance trade-off can be tailored to match specific application requirements. This multi-resolution nature allows for extraction of compute-efficient high-performing models during inference, with the same parameter space. To validate LookupViT’s efficacy, we show results on multiple benchmarks like image and video classification, and image captioning. Notably, due to the information bottleneck, LookupViT also shows out-of-the-box robustness to image corruptions. The key contributions of this work are -



**Fig. 2: Bidirectional information flow in LookupViT block.** LookupViT restricts the heavy computation to the compressed tokens, while extracting information from the lookup tokens. The lookup tokens then update themselves by reusing the information exchange computation.

- **Efficient Token Compression:** LookupViT introduces a novel Multi-Head Bidirectional Cross-attention (MHBC) module that enables effective information flow with significant computational savings.

- **Generalized Framework:** LookupViT offers a flexible framework applicable to visual modalities. It also offers compute-performance trade-offs via multi-resolution ability of compressed tokens, with identical model parameters.
- **Enhanced Performance:** LookupViT generalizes well to applications on image/video modalities, and boasts out-of-the box robustness to corruptions.

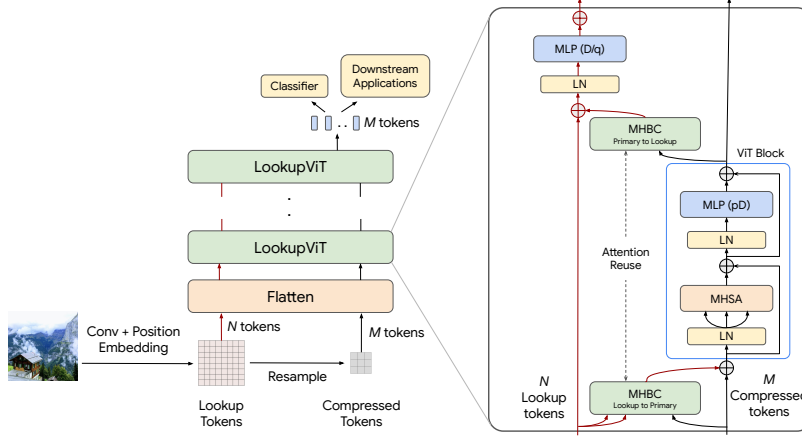
## 2 Related Works

Since the introduction of the Vision Transformer (ViT), a multitude of works have endeavored to improve its efficiency and scalability.

**Multi-scale and Hierarchical Features:** Early studies such as [13, 22, 30] utilized non-overlapping patches with multi-scale or hierarchical features, achieving notable success in both image and video domains [1]. Concurrently, [24] proposed hierarchical designs for efficient training and inference across these modalities. These approaches pushed accuracy boundaries, but often at the expense of added architectural complexity. For instance, MViTv2 [21] decomposes relative position embedding and residual pooling, while CSWin [11] integrates cross-shaped windows within a hierarchical framework. This creates a trade-off between enhanced accuracy and the potential loss of ViT’s inherent simplicity and scalability. LookupViT’s compressed and lookup tokens has some parallels with the convolution-based OctConv’s [8] low and high frequency features. However LookupViT restricts heavy processing to compressed tokens, and enjoys scalability of Transformers.

**Token Merging and Sampling:** Another prominent research direction involves token merging and pruning. [5, 14, 26, 31] aim to reduce redundant tokens through merging, sampling, or pruning. For example, [5] uses similarity to groups and merge tokens, while [14] employs adaptable token sampling. While valuable, these techniques often introduce heuristics and generally function as post-processing steps. Furthermore, they can face challenges when extending to modalities beyond images, such as videos or multi-modal data. In contrast, LookupViT emphasizes intrinsic compression through its core architecture, replacing the ViT block. Importantly, LookupViT remains harmonious with the potential application of token merging or sampling for further optimization.

**Token compression:** Instead of merging tokens, [23] learns a smaller number of  $M$  patches from the original  $N$  patches in ViT using a learnable weight matrix. Similarly, TokenLearner [25] compressed all ViT tokens into a smaller set of 8-16 tokens and performing self-attention within this reduced set, but after a certain number of vanilla ViT layers. Perceiver [19] proposes learning a small set of tokens directly from the pixel space using iterative unidirectional cross-attention. These two methods are most closely related to our work. However, TokenLearner’s compression achieves optimal performance only when processing at least 50 – 75% of the network with ViT blocks, leading to no reduction in computation for a significant number of layers. In contrast, LookupViT can be trained entirely



**Fig. 3: LookupViT Architecture:** The LookupViT block is stacked multiple times similar to vanilla ViT. Each LookupViT block has two parallel computation streams for the two different types of tokens. Heavy computation happens on a fixed smaller number of compressed tokens, while light computation happens on the much higher number of lookup tokens. There is an asynchronous information exchange between the two token sets using the Multi-Head Bi-Directional Cross Attention (MHBC) block.

with lookup blocks, reducing computational complexity without compromising performance. Furthermore, unlike Perceiver [19], which uses unidirectional pixel-level cross-attention, LookupViT operates on tokens with bi-directional cross-attention to update both compressed and lookup tokens.

**Flexible patch and resolution:** Recent works like FlexiViT [3] address the fixed patch size limitation by training with multiple patch sizes, enabling ViT to scale across different patch sizes and image resolutions. Na-ViT [9] explores sequence packing to train images with arbitrary resolution and aspect ratio, allowing inference on any resolution image. Analogous to these works, we show that LookupViT can also be trained with varying compression ratios to obtain multiple models during inference with the same parameter space.

### 3 LookupViT Methodology

In this section, we discuss the LookupViT framework in detail, starting with a high-level architectural discussion, and then focusing on specific design choices. We also discuss its applicability to downstream tasks and Multi-Resolution flexibility. We conclude this section with an analysis of the improved computational complexity.

#### 3.1 Overall Architecture

An overview of the LookupViT architecture is presented in Figure 3. Similar to the ViT architecture, it comprises of a stack of LookupViT blocks. First, an input

RGB image (or video) is divided into non-overlapping patches. These patches are then passed through a convolutional layer to generate feature embeddings. Positional embeddings are then added to construct the input tokens – a process identical to the standard ViT architecture [12]. Unlike vanilla ViT, the core idea here is - *to compress visual information into a smaller number of tokens, focusing heavy computation exclusively on those tokens.*

A fixed number of tokens  $M$  ( $\ll N$ ), which we name as the *compressed* tokens are sampled from the input tokens, using bilinear interpolation. Computationally intensive processing is performed on the compressed tokens, analogous to a standard ViT block, while exchanging information with the original tokens through asynchronous Multi-Head Bidirectional Cross-Attention (MHBC). The process unfolds as follows - (1) **Information Gathering:** Compressed tokens use cross-attention to “look” at the original tokens (termed *lookup* tokens) and gather relevant information. (2) **Representation Refinement:** Compressed tokens exchange information amongst themselves, updating their representations. (3) **Global Context Infusion:** The lookup tokens utilize the processed, information-rich compressed tokens, to update their own representations, reusing the attention weights calculated during Information Gathering for efficiency.

During this entire process, the lookup tokens are forced to gather information only by interacting with the compressed tokens, thus reducing computational complexity. Additionally, the lookup tokens pass through a MLP block with a smaller projection dimension ( $D/q$ ) compared to the vanilla model projection ( $pD$ ), which is applied on the compressed tokens, where  $D$  represents the transformer embedding dimension ( $(p, q) = (4, 2)$ ). This optimization further reduces computations. The LookupViT block’s ability to achieve performance comparable to the baseline, despite this substantial MLP bottleneck, demonstrates the effectiveness of the information exchange between compressed and lookup tokens.

### 3.2 Input Tokenization

The construction of lookup token embeddings similar to standard ViT [12] tokenization strategy. Given an input image  $\mathbf{X} \in \mathbb{R}^{h \times w \times c}$ , it is passed through a convolutional layer to obtain lookup features  $\mathbf{F}_l \in \mathbb{R}^{h_l \times w_l \times D}$ . A learnable lookup positional embedding  $\mathbf{F}_{l,pos} \in \mathbb{R}^{h_l \times w_l \times D}$  is added to this feature map. These tokens are then significantly downsampled to a fixed shape –  $(h_p, w_p)$ , which constitute the compressed tokens. This can be summarized as below -

$$\mathbf{F}_p \leftarrow \mathcal{T}(\mathbf{F}_l, (h_p, w_p)) \quad \mathbf{F}_{p,pos} \leftarrow \mathcal{T}(\mathbf{F}_{l,pos}, (h_p, w_p)) \quad (1)$$

$$\mathbf{F}_p \leftarrow \mathbf{F}_p + \mathbf{F}_{p,pos} \quad \mathbf{F}_l \leftarrow \mathbf{F}_l + \mathbf{F}_{l,pos} \quad (2)$$

The operator  $\mathcal{T}(\mathbf{x}, s)$  bilinearly resizes  $\mathbf{x}$  to shape  $s$ . The lookup and compressed token grids have sizes  $(h_l, w_l)$  and  $(h_p, w_p)$ , and  $D$  is the embedding dimension. These feature maps  $\mathbf{F}_p$  and  $\mathbf{F}_l$  are then spatially flattened to  $\mathbf{z}_p^0$  and  $\mathbf{z}_l^0$ :

$$\mathbf{z}_p^0 = [\mathbf{F}_{p(0,0)}, \dots, \mathbf{F}_{p(h_p-1, w_p-1)}] \quad \mathbf{z}_p^0 \in \mathbb{R}^{h_p \cdot w_p \times D} \quad (3)$$

$$\mathbf{z}_l^0 = [\mathbf{F}_{l(0,0)}, \dots, \mathbf{F}_{l(h_l-1, w_l-1)}] \quad \mathbf{z}_l^0 \in \mathbb{R}^{h_l \cdot w_l \times D} \quad (4)$$

These flattened feature maps  $\mathbf{z}_p^0$  and  $\mathbf{z}_l^0$  (compressed and lookup tokens respectively) are passed as input to the LookupViT block, which efficiently refines these representations through information exchange, as explained in Section 3.3. The resize ratio  $C = h_l.w_l/h_p.w_p$  is a flexibility parameter, representing the degree of information compression. This enables us to flexibly train the model with varying resize ratio, thus allowing compute-aware model extraction with a specific  $C$ . A smaller value for  $C$  indicates more number of compressed tokens and thus better representation power. In fact,  $C = 1$  would represent the vanilla ViT with certain extra computations due to the cross-attention. We denote the number of lookup and compressed tokens by  $N = h_l.w_l$  and  $M = h_p.w_p$  respectively. This form of tokenization can readily extend to videos, where a third dimension representing time would be introduced. The compression ratio would then become  $C = h_l.w_l.t_l/h_p.w_p.t_p$ , where  $t$  denotes the temporal dimension.

### 3.3 LookupViT Block

The  $k^{th}$  LookupViT block consumes the compressed tokens  $\mathbf{z}_p^{k-1}$  and lookup tokens  $\mathbf{z}_l^{k-1}$  from its previous block, facilitates information exchange between the two token sets, and passes the updated representations to the next block. The novel architectural design here is the asynchronous Multi-Head Bidirectional Cross-attention (MHBC). Intuitively, in the first layer, the lookup tokens maintain a richer image representation than the compressed tokens. However, after multiple passes through the LookupViT block, the compressed tokens accumulate relevant compressed image information, thus making them suitable for downstream tasks. This happens through iterative communication between the lookup and compressed tokens in every LookupViT block (Algorithm 4). This can be summarized into three key steps -

**Information Gathering:** In this step, there is a unidirectional information flow from the lookup to the compressed tokens through  $\text{MHBC}_{l \rightarrow p}$ . The compressed tokens are used as query ( $\mathbf{Q}$ ) and lookup tokens as key-value ( $\mathbf{K}, \mathbf{V}$ ). Algorithm 1 presents this part of the proposed MHBC module. Additionally, we store the attention weights  $\mathcal{A}$  computed in this step to be re-used while sharing information in the reverse direction.

**Representation Refinement:** After the information extraction step, the compressed tokens go through a vanilla ViT block (self-attention followed by MLP), as illustrated in Algorithm 3. The MLP dimension upscaling factor  $p$  is kept equal to 4, as in vanilla ViT. But this computation happens on the smaller compressed token set. This step allows internal information sharing between compressed tokens to update their representation.

**Global Context Infusion:** The information gathering along with the ViT based processing enriches the compressed token features, as they contain a compressed global representation of the image. While the lookup tokens do not directly share information amongst themselves, they are notified about the global information

through a reverse direction information exchange, from compressed to lookup tokens, as depicted in Algorithm 2 (MHBC<sub>*l*→*p*</sub>). Rather than recomputing the attention matrix, we reuse the attention matrix previously saved in MHBC<sub>*p*→*l*</sub>. This relation further imposes implicit similarity constraints between the two feature maps, and enhances information exchange. Finally, to refine the lookup features, we apply a low dimensional MLP block, with a dimension  $(D/q)$ ,  $pq$  times smaller than the vanilla ViT MLP dimension (we set  $(p, q) = (4, 2)$  in all our experiments). This enriches the lookup tokens for information extraction by the compressed tokens in the next LookupViT block.

---

**Algorithm 1** MHBC<sub>*l*→*p*</sub>


---

**In:**  $\mathbf{z}_p \in \mathbb{R}^{M \times D}$ ;  $\mathbf{z}_l \in \mathbb{R}^{N \times D}$   
 1:  $Q \leftarrow \text{LN}(w_Q \mathbf{z}_p)$   
 2:  $K \leftarrow \text{LN}(w_K \mathbf{z}_l)$   
 3:  $V \leftarrow w_V \mathbf{z}_l$   
 4:  $\mathcal{A} \leftarrow \text{softmax}(QK^T)$   $\triangleright \mathcal{A} \in \mathbb{R}^{M \times N}$   
 5:  $\mathbf{z}_p \leftarrow \mathbf{z}_p + \mathcal{A}V$   
**Return**  $\mathbf{z}_p, \mathcal{A}$

---



---

**Algorithm 2** MHBC<sub>*p*→*l*</sub>


---

**In:**  $\mathbf{z}_l \in \mathbb{R}^{N \times D}$ ;  $\mathbf{z}_p \in \mathbb{R}^{M \times D}$ ;  $\mathcal{A} \in \mathbb{R}^{M \times N}$   
 1:  $V \leftarrow \text{LN}(w_V \mathbf{z}_p)$   
 2:  $\triangleright$  Layer Norm on Values  
 3:  $\mathbf{z}_l \leftarrow \mathbf{z}_l + \mathcal{A}^T V$   
 4:  $\triangleright$  Reuse pre-computed weights  
**Return**  $\mathbf{z}_l$

---



---

**Algorithm 3** ViTBlock
 

---

**In:**  $\mathbf{z}_p \in \mathbb{R}^{M \times D}$ ;  $p \in \mathbb{N}$   
 1:  $\mathbf{z}_p \leftarrow \mathbf{z}_p + \text{MHSA}(\text{LN}(\mathbf{z}_p))$   
 2:  $\triangleright$  Multi-Head Self-Attention  
 3:  $\mathbf{z}_p \leftarrow \mathbf{z}_p + \text{MLP}(\text{LN}(\mathbf{z}_p); pD)$   
**Return**  $\mathbf{z}_p$

---



---

**Algorithm 4** LookupViTBlock
 

---

**In:**  $\mathbf{z}_p \in \mathbb{R}^{M \times D}$ ;  $\mathbf{z}_l \in \mathbb{R}^{N \times D}$ ;  $p, q \in \mathbb{N}$   
 1:  $\mathbf{z}_p, \mathcal{A} \leftarrow \text{MHBC}_{l \rightarrow p}(\text{LN}(\mathbf{z}_p), \text{LN}(\mathbf{z}_l))$   
 2:  $\mathbf{z}_p \leftarrow \text{ViTBlock}(\mathbf{z}_p, p)$   
 3:  $\mathbf{z}_l \leftarrow \mathbf{z}_l + \text{MHBC}_{p \rightarrow l}(\text{LN}(\mathbf{z}_p), \text{LN}(\mathbf{z}_l), \mathcal{A})$   
 4:  $\mathbf{z}_l \leftarrow \mathbf{z}_l + \text{MLP}(\text{LN}(\mathbf{z}_l); D/q)$   
**Return**  $\mathbf{z}_p, \mathbf{z}_l$

---

**Multi-Resolution Tokens:** The compressed tokens are constructed by simply resizing the lookup tokens in a non-learnable fashion. Hence, it is possible to share the same parameter space and lookup tokens while having multiple compressed token resolutions. To do this, we choose the compressed token size uniformly at random during training, seeking inspiration from FlexiViT [3]. Once trained in this fashion, we can extract multiple high performing models having different computational requirements from a single trained model. This flexibility makes our method utilizable in a variety of settings, depending on resource availability.

### 3.4 Training and Token Utilization for Downstream Applications

In LookupViT, we maintain two sets of tokens throughout the network -  $N$  lookup tokens and  $M$  compressed tokens. For classification, we can apply the classifier to either or both token sets. Empirically, we’ve found that enforcing classification loss on both heads yields the best performance. We use global average pooling on the respective token sets, followed by two separate classifiers. The joint loss function is then optimized with equal weights.

Although the training loss is applied independently to both token sets, we find that during inference, the classifier on the compressed tokens is sufficient. However,

adding the classifier output from the lookup tokens does improve performance marginally. Since there is no added computational cost for classification, we average the outputs of both compressed and lookup heads with equal weights. For downstream applications beyond classification (e.g., vision-language model tasks like captioning), a decoder is used on the LookupViT encoder. In such cases, using a limited compressed token set computationally benefits the cross-attention block. Hence, we experiment using only the compressed tokens.

### 3.5 Computational Complexity

Let  $\mathcal{C}_x$  denote the computation of a procedure  $x$ . Then, given the feature dimension  $D$ , number of lookup tokens  $N$ , number of compressed tokens  $M (<< N)$ , MLP upscaling factor  $p = 4$  (on compressed tokens) and downscaling factor  $q = 2$  (on lookup tokens), the computational complexity of the vanilla ViT and LookupViT blocks can be represented as follows (neglecting smaller terms).

$$\mathcal{C}_{\text{ViT}} = 2N^2D + 12ND^2 \quad (5)$$

$$\mathcal{C}_{\text{LookupViT}} = (3NM + 2M^2)D + (4N + 15M)D^2 \quad (6)$$

Notice that we get rid of the quadratic dependence on the number of lookup tokens  $N$  and reduce the attention and linear projection computations individually. Since the number of compressed tokens  $M (<< N)$  stay constant at a user-specified value, the attention reduction factor grows quickly, enabling scalability for usage at higher resolutions. Typically, for an image resolution of 384, we use  $N = 576$  and  $M = 25$ , which shows superior performance than the vanilla model, while simultaneously reducing FLOPs by a factor greater than 3.

## 4 Results

**Implementation Details:** As ViTs are prone to overfit more as compared to CNNs, they either need pre-training on large datasets like JFT [28] or augmentation based training frameworks like DeIT [29] or AugReg [27]. Due to the ease of implementation and adaptability to other tasks that we pursue in this work, we build our implementation on top of [27]. We implement LookupViT in JAX [6] within the Big Vision repository [4]. We adopt the exact training settings as in [27] (like learning rate, training epochs, etc) without performing any parameter sweeps. We also train TokenLearner [25], another state-of-the-art token compression technique, on the same repository for fair comparison, with 16 tokens for all experiments. TokenLearner<sub>1/2</sub> denotes their compression module is applied half-way through the network, which the authors recommend.

**Image classification:** We evaluate LookupViT on image classification while – (a) training from scratch on ImageNet-1k [10], and (b) finetuning on ImageNet-1k from a ImageNet-21k pre-trained model. The popular benchmark ImageNet-1k has 1.28 million training images and 50,000 validation images across 1,000

**Table 1:** Comparison with state-of-the-art methods on ImageNet-1k and ImageNet-21K pre-training followed by ImageNet-1K finetuning.

| Variant | Method                           | ImageNet-1K | ImageNet-21K | GFLOPs |
|---------|----------------------------------|-------------|--------------|--------|
| B/16    | ViT [12]                         | 78.6        | 83.7         | 35.1   |
|         | TokenLearner <sub>1/2</sub> [25] | 75.7        | 82.0         | 19.1   |
|         | TokenLearner <sub>3/4</sub> [25] | 77.5        | 82.9         | 27.1   |
|         | Perceiver [19]                   | 78.0        | -            | 707    |
|         | LookupViT <sub>3×3</sub>         | 77.1        | 77.9         | 12.9   |
|         | LookupViT <sub>5×5</sub>         | 79.1        | 81.6         | 16.5   |
|         | LookupViT <sub>7×7</sub>         | 79.7        | 83.0         | 21.9   |
|         | LookupViT <sub>10×10</sub>       | 80.2        | 83.9         | 33.6   |
| L/16    | ViT [12]                         | 75.7        | 84.0         | 123.5  |
|         | TokenLearner <sub>1/2</sub> [25] | 78.5        | 84.2         | 66.5   |
|         | TokenLearner <sub>3/4</sub> [25] | 77.6        | 84.6         | 94.6   |
|         | LookupViT <sub>3×3</sub>         | 77.2        | 78.4         | 46.2   |
|         | LookupViT <sub>5×5</sub>         | 78.7        | 81.9         | 58.8   |
|         | LookupViT <sub>7×7</sub>         | 79.1        | 83.3         | 77.7   |
|         | LookupViT <sub>10×10</sub>       | 79.2        | 84.1         | 118.5  |

categories. ImageNet-21k has 12.8 million images across 21,000 categories. For all experiments, we train and report performance on the validation set with an image size of  $224 \times 224$ , unless specified otherwise. We experiment with two model sizes  $B$  and  $L$ , with model parameters as defined in ViT [12].

We present the results on image classification in Table 1. LookupViT is flexible enough to offer multiple models with the same parameter, by varying the compression factor  $C$ , the ratio between the number of lookup and compressed tokens. Results indicate that training from scratch with the B/16 model on ImageNet-1k, LookupViT<sub>5×5</sub> performs better than ViT with  $2.12\times$  lesser FLOPs. LookupViT<sub>10×10</sub> outperforms ViT by 1.6% while still being computationally cheaper. It also offers similar gains compared to TokenLearner and Perceiver. On image size of 384, we can see in Figure 1b, LookupViT offers more than  $3\times$  computational gains compared to ViT. The figure also shows that LookupViT computationally scales quite efficiently w.r.t. ViT when the image size is increased.

The performance of LookupViT on the large model is even better when trained from scratch. Even with  $3 \times 3$  compressed tokens, LookupViT performs much better than ViT, requiring  $2.67\times$  lower FLOPs. It is interesting to note that we did observe instabilities while training large models for both ViT and TokenLearner, whereas we do not observe such instabilities in LookupViT. When using ImageNet-21k pretrained models for ImageNet-1k finetuning, we achieve higher accuracy than the ViT models using our  $10 \times 10$  models, while still maintaining lesser computational requirements.

**Analyzing the robustness of LookupViT:** Vision models often exhibit surprising vulnerability to image corruptions and adversarial perturbations. While the ViT architecture is more robust than CNNs in general [2], we explore out-of-the box robustness of LookupViT to image corruptions and adversarial settings, without including any additional robustness losses, augmentations or training strategies. We evaluate on ImageNet-A,C,O,R [16–18]. (see Appendix A1)

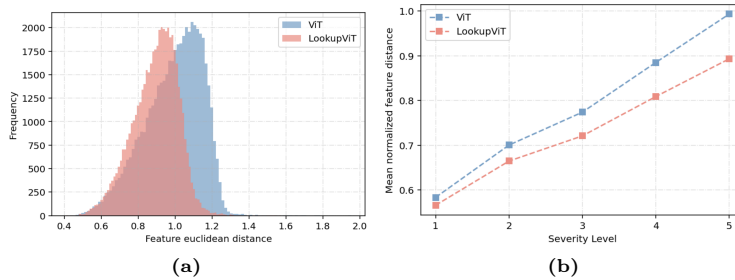
**Table 2:** Robustness on ImageNet-A, C, O, R [16–18] datasets.

| Variant | Method                           | ImageNet-C  | ImageNet-O  | ImageNet-R  | ImageNet-A |
|---------|----------------------------------|-------------|-------------|-------------|------------|
| B/16    | ViT [12]                         | 56.4        | 24.6        | 25.8        | 6.7        |
|         | TokenLearner <sub>3/4</sub> [25] | 55.7        | 24.5        | 23.1        | 5.7        |
|         | LookupViT <sub>5×5</sub>         | 56.4        | 24.0        | 25.9        | 5.4        |
|         | LookupViT <sub>7×7</sub>         | 58.4        | 24.9        | 28.0        | 7.6        |
|         | LookupViT <sub>10×10</sub>       | <b>59.9</b> | <b>25.7</b> | <b>29.1</b> | <b>8.1</b> |
| L/16    | ViT [12]                         | 55.3        | 22.4        | 19.4        | 3.5        |
|         | TokenLearner <sub>3/4</sub> [25] | 55.6        | 20.3        | 24.0        | 3.7        |
|         | LookupViT <sub>5×5</sub>         | 58.4        | 25.7        | 26.5        | 7.0        |
|         | LookupViT <sub>7×7</sub>         | 59.7        | <b>26.7</b> | 28.0        | 8.3        |
|         | LookupViT <sub>10×10</sub>       | <b>60.4</b> | 26.5        | <b>28.1</b> | <b>8.4</b> |

As shown in Table 2, LookupViT performs better than ViT across the board, showing robust performance on natural, unmodified images (ImageNet-A), resilience to image corruptions (ImageNet-C) and artistic renditions (ImageNet-R), and strong generalization to out-of-distribution samples (ImageNet-O). These unanimously suggests that LookupViT’s mechanism of extracting only useful information inherently improves its ability to handle noisy or distorted inputs.

We further investigate LookupViT’s robustness to perturbations by comparing the image-wise normalized feature deviation due to corruptions in Figure 4, on the ImageNet-C dataset. It also shows that the margin of improvement of LookupViT over ViT goes up as we increase the corruption severity. This shows the model’s robust behaviour beyond its better discriminatory ability.

**Using pre-trained LookupViT for Captioning:** In Table 3, we assess the capability of LookupViT as a pre-trained model to judge its transferability for other tasks. We investigate its performance on image captioning using Locked Image Tuning (LiT) style training [32]. Following LiT, we freeze the parameters of the pretrained LookupViT image encoder, which is pre-trained on ImageNet-21k. We then introduce a simple text decoder, initialized randomly, and train it to generate captions corresponding to the image representations produced by LookupViT. We perform this experiment on the COCO Captions [7] dataset.



**Fig. 4:** (a) Density of normalized feature distance for severity=5 over all corruptions. (b) Mean normalized feature distance over all corruptions for different severity.

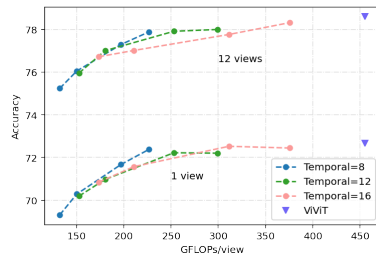
**Table 3:** Image captioning on COCO-Captions [7] using LiT decoder style training [32] with frozen encoder. (LookupViT: LViT, TokenLearner: TL)

|       | ViT [12] | TL <sub>1/2</sub> [25] | LViT <sub>3×3</sub> | LViT <sub>5×5</sub> | LViT <sub>7×7</sub> | LViT <sub>10×10</sub> |
|-------|----------|------------------------|---------------------|---------------------|---------------------|-----------------------|
| Cider | 112.6    | 110.3                  | 104.3               | 108.4               | 111.3               | 111.7                 |
| FLOPS | 59.5     | 38.2                   | 31.9                | 35.9                | 42.1                | 55.3                  |

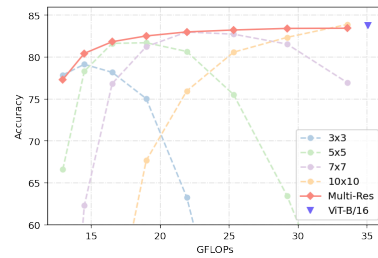
LookupViT with  $7 \times 7$  compressed tokens exhibits similar performance to ViT, even without finetuning, thus offering significant reduction in FLOPs. This highlights the quality of visual representations learned by LookupViT and its potential as a versatile backbone for various vision-and-language tasks.

**Video classification:** LookupViT can be easily extended to videos. We modify the ViViT [1] spatio-temporal B/16 encoder to construct LookupViViT. As in ViViT, the initial Conv3D layer, with kernel size  $16 \times 16 \times 2$  operates on a video of  $224 \times 224 \times 3 \times 32$ . The resultant 3D tokens serve as lookup tokens, which are bilinearly downsampled to obtain the spatio-temporal compressed tokens. After flattening the tokens, the rest follows exactly as in LookupViT.

We carry out experiments on the Kinetics400 [20] and SomethingSomethingV2 (SSv2) [15] datasets. Kinetics has 240k videos of 10 second duration each. Being a dynamic YouTube based dataset, it often incurs data loss due to deletion, so we could only train on a subset of videos which ViViT was trained on at the time of its paper publication, leading to lower baselines. SSv2 has 220k videos of 2-6 second duration each. On SSv2, ViViT [1] does not report numbers using the Spatio-Temporal B/16 model, but we follow the training recipe as mentioned for their L/16 Factorized Encoder [1], and initialize our LookupViViT as well as ViViT models from their corresponding Kinetics400 finetuned models.



(a) LViViT with different compressed tokens



(b) Multi-resolution compressed tokens

**Fig. 5:** (a) Video classification on K400 with different spatio-temporal compressed tokens for LookupViViT (LVViT). Color denotes the number of temporal token and points on each curve are with increasing number of spatial tokens. (b) Training a single model (“Multi-Res”) which can handle different number of compressed tokens, offering compute-performance trade-off with the same parameter space. The other models are trained individually but evaluated at all resolutions.

**Table 4:** Comparison of LookupViT-B/16 based ViViT [1] model with state-of-the-art methods on Kinetics400 [20] and SomethingSomethingV2 [15].

| Method                        | Kinetics400 |          | SomethingSomethingV2 |          | GFLOPs/view |
|-------------------------------|-------------|----------|----------------------|----------|-------------|
|                               | 1 view      | 12 views | 1 view               | 12 views |             |
| ViViT [12]                    | 72.6        | 78.6     | 51.2                 | 52.3     | 455.1       |
| LookupViViT <sub>9×9×8</sub>  | 72.4        | 77.9     | 56.3                 | 58.3     | 226.5       |
| LookupViViT <sub>8×8×16</sub> | 72.5        | 77.8     | 57.0                 | 59.2     | 311.2       |
| LookupViViT <sub>9×9×16</sub> | 72.5        | 78.3     | 57.1                 | 59.6     | 375.8       |

**Table 5: Dissecting the network:** Component-wise performance impact

| Models                                            | GFLOPS | Accuracy |
|---------------------------------------------------|--------|----------|
| LookupViT-B/16                                    | 16.5   | 79.1     |
| – No Lookup Tokens                                | 4.5    | 69.2     |
| – No MHBC <sub><math>p \rightarrow l</math></sub> | 11.0   | 69.7     |
| – No Loss on Lookup Tokens                        | 16.5   | 78.2     |
| – No Loss on Primary Tokens                       | 16.5   | 78.4     |
| Random Primary Tokens                             | 16.5   | 77.9     |

The results on video classification are presented in Table 4 for both single and multi-crop evaluation following ViViT [1]. Moreover, we report results for various number of spatial and temporal compressed tokens and plot them in Figure 5a for Kinetics400. LookupViViT models with half the FLOPs as ViViT show competitive results on Kinetics400. We also found the trends in accuracy to be increasing with the number of spatial and/or temporal tokens, as expected.

Interestingly, as in Table 4, LookupViViT performs significantly better than ViViT on SSv2. We observe a 5% – 6% improvement in accuracy with half the FLOPs. This further bolsters LookupViT’s robustness claim, as in SSv2 backgrounds and objects are similar across classes, thus needing recognition of fine-grained motion [1], which our model does better than ViViT. (See Appendix A2, A5)

**Multi-resolution LookupViT:** We empirically demonstrate the effectiveness of LookupViT’s multi-resolution tokenization. By varying the downsampling ratio, we can control the trade-off between computational cost and representation capacity while keeping the parameter size constant. Inspired by [3], during training, we randomly choose the compressed token resolution, ranging from  $3 \times 3$  to  $10 \times 10$  for every batch of data. We call this model “Multi-Res”. The number of lookup tokens is always kept fixed at  $14 \times 14$  for an image resolution of  $224 \times 224$ . To highlight its efficacy, we also train individual models with the designated number of compressed tokens, while evaluating at all compressed resolutions.

We present results in Figure 5b, where all models are pretrained on ImageNet-21k and finetuned on ImageNet-1k. For Multi-Res, both steps are carried out with a multi-resolution training technique. As we can see, the performance of the Multi-Res model is remarkably close to that of the individual models at the points for which they are trained. The performance of the individual models do not hold up when evaluated at other resolutions. This finding highlights the

adaptability of LookupViT and its potential to streamline model selection by offering performance-computation trade-offs within a single trained architecture.

## 5 Ablations

The ablations performed in this section use a B/16 model with a compressed token size  $5 \times 5$  trained from scratch on ImageNet-1k. This model, with all the components in place, reaches a top-1 classification accuracy of 79.1. We discuss the component-wise importance of LookupViT model in Table 5.

**No Lookup Tokens:** We consider constructing the compressed tokens by aggressively downsampling the image features through convolution, while having no information support through the higher resolution lookup tokens, i.e. no  $\text{MHBC}_{l \rightarrow p}$  or  $\text{MHBC}_{p \rightarrow l}$ . The compressed tokens go through only the vanilla ViT. This leads to much lower performance, indicating that ViT, by itself, doesn’t work well with very limited tokens without additional information exchange.

**No  $\text{MHBC}_{p \rightarrow l}$ :** From the previous step, we now add  $\text{MHBC}_{l \rightarrow p}$ , which facilitates information transfer from lookup to compressed tokens, while still not updating the lookup tokens. This leads to a slight increase in performance as compared to the previous setup, as the lookup tokens are not updated at all with global information. However, this step involves construction of compressed tokens using a parameter-free resize rather than a convolutional downsampling, which enables use of same model across different compressed token sizes.

**No Lookup/Compressed Loss:** Next, we add  $\text{MHBC}_{p \rightarrow l}$ , a source of information exchange from the compressed to the lookup tokens, with loss computation still only on the compressed tokens. This leads to a further 8.5% increase in accuracy, thus justifying the need for the bidirectional MHBC. We also consider the case where we add loss on the lookup logits only, but not on the compressed, and this leads to an equivalent performance, indicating the near equal capability of compressed and lookup tokens.

**Random Compressed Tokens:** We also experiment with random learnable compressed tokens in the first layer, instead of resizing from lookup tokens. We observe a  $\sim 1\%$  performance drop, thus showing the effectiveness of parameter-free resize operation for constructing compressed tokens.

## 6 Conclusions

In this work, we present a novel LookupViT architecture, which efficiently compresses sparse and redundant visual information to fewer tokens. By efficiently combining lower and higher resolution tokens with bidirectional cross-attention, LookupViT achieves a significant reduction in FLOPs while upholding performance of ViT. Its effectiveness is demonstrated on diverse vision tasks, like image and video classification, image captioning, as well as its generalizability and robustness to visual corruptions.

Future work includes extending our model to dense prediction tasks like object detection and semantic segmentation, as well as scaling to larger model sizes.

## References

1. Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., Schmid, C.: Vivit: A video vision transformer. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 6836–6846 (2021)
2. Bai, Y., Mei, J., Yuille, A.L., Xie, C.: Are transformers more robust than cnns? Advances in neural information processing systems **34**, 26831–26843 (2021)
3. Beyer, L., Izmailov, P., Kolesnikov, A., Caron, M., Kornblith, S., Zhai, X., Minderer, M., Tschannen, M., Alabdulmohsin, I., Pavetic, F.: Flexivit: One model for all patch sizes. CVPR (2023)
4. Beyer, L., Zhai, X., Kolesnikov, A.: Big vision. [https://github.com/google-research/big\\_vision](https://github.com/google-research/big_vision) (2022)
5. Bolya, D., Fu, C.Y., Dai, X., Zhang, P., Feichtenhofer, C., Hoffman, J.: Token merging: Your vit but faster. ICLR (2023)
6. Bradbury, J., Frostig, R., Hawkins, P., Johnson, M.J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., Zhang, Q.: JAX: composable transformations of Python+NumPy programs (2018), <http://github.com/google/jax>
7. Chen, X., Fang, H., Lin, T., Vedantam, R., Gupta, S., Dollár, P., Zitnick, C.L.: Microsoft COCO captions: Data collection and evaluation server. CoRR (2015)
8. Chen, Y., Fan, H., Xu, B., Yan, Z., Kalantidis, Y., Rohrbach, M., Yan, S., Feng, J.: Drop an octave: Reducing spatial redundancy in convolutional neural networks with octave convolution. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 3435–3444 (2019)
9. Dehghani, M., Mustafa, B., Djolonga, J., Heek, J., Minderer, M., Caron, M., Steiner, A., Puigcerver, J., Geirhos, R., Alabdulmohsin, I.M., et al.: Patch n’pack: Navit, a vision transformer for any aspect ratio and resolution. Advances in Neural Information Processing Systems **36** (2024)
10. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE conference on computer vision and pattern recognition. pp. 248–255. Ieee (2009)
11. Dong, X., Bao, J., Chen, D., Zhang, W., Yu, N., Yuan, L., Chen, D., Guo, B.: Cswin transformer: A general vision transformer backbone with cross-shaped windows. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 12124–12134 (2022)
12. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. ICLR (2020)
13. Fan, H., Xiong, B., Mangalam, K., Li, Y., Yan, Z., Malik, J., Feichtenhofer, C.: Multiscale vision transformers. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 6824–6835 (2021)
14. Fayyaz, M., Koochpayegani, S.A., Jafari, F.R., Sengupta, S., Joze, H.R.V., Sommerlade, E., Pirsiavash, H., Gall, J.: Adaptive token sampling for efficient vision transformers. In: European Conference on Computer Vision. pp. 396–414. Springer (2022)
15. Goyal, R., Ebrahimi Kahou, S., Michalski, V., Materzynska, J., Westphal, S., Kim, H., Haenel, V., Freund, I., Yianilos, P., Mueller-Freitag, M., et al.: The "something something" video database for learning and evaluating visual common sense. In: Proceedings of the IEEE international conference on computer vision. pp. 5842–5850 (2017)

16. Hendrycks, D., Basart, S., Mu, N., Kadavath, S., Wang, F., Dorundo, E., Desai, R., Zhu, T., Parajuli, S., Guo, M., Song, D., Steinhardt, J., Gilmer, J.: The many faces of robustness: A critical analysis of out-of-distribution generalization. ICCV (2021)
17. Hendrycks, D., Dietterich, T.: Benchmarking neural network robustness to common corruptions and perturbations. ICLR (2019)
18. Hendrycks, D., Zhao, K., Basart, S., Steinhardt, J., Song, D.: Natural adversarial examples. CVPR (2021)
19. Jaegle, A., Gimeno, F., Brock, A., Zisserman, A., Vinyals, O., Carreira, J.: Perceiver: General perception with iterative attention. PMLR (2021)
20. Kay, W., Carreira, J., Simonyan, K., Zhang, B., Hillier, C., Vijayanarasimhan, S., Viola, F., Green, T., Back, T., Natsev, P., et al.: The kinetics human action video dataset. arXiv preprint arXiv:1705.06950 (2017)
21. Li, Y., Wu, C.Y., Fan, H., Mangalam, K., Xiong, B., Malik, J., Feichtenhofer, C.: Mvitv2: Improved multiscale vision transformers for classification and detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 4804–4814 (2022)
22. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin transformer: Hierarchical vision transformer using shifted windows. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 10012–10022 (2021)
23. Renggli, C., Pinto, A.S., Houlsby, N., Mustafa, B., Puigcerver, J., Riquelme, C.: Learning to merge tokens in vision transformers. arXiv preprint arXiv:2202.12015 (2022)
24. Ryali, C., Hu, Y.T., Bolya, D., Wei, C., Fan, H., Huang, P.Y., Aggarwal, V., Chowdhury, A., Poursaeed, O., Hoffman, J., et al.: Hiera: A hierarchical vision transformer without the bells-and-whistles. arXiv preprint arXiv:2306.00989 (2023)
25. Ryoo, M.S., Piergiovanni, A., Arnab, A., Dehghani, M., Angelova, A.: Tokenlearner: What can 8 learned tokens do for images and videos? NeurIPS (2021)
26. Song, Z., Xu, Y., He, Z., Jiang, L., Jing, N., Liang, X.: Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction. arXiv preprint arXiv:2203.04570 (2022)
27. Steiner, A., Kolesnikov, A., Zhai, X., Wightman, R., Uszkoreit, J., Beyer, L.: How to train your vit? data, augmentation, and regularization in vision transformers. arXiv preprint arXiv:2106.10270 (2021)
28. Sun, C., Shrivastava, A., Singh, S., Gupta, A.: Revisiting unreasonable effectiveness of data in deep learning era. In: Proceedings of the IEEE international conference on computer vision. pp. 843–852 (2017)
29. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers & distillation through attention. In: International conference on machine learning. pp. 10347–10357. PMLR (2021)
30. Wang, W., Xie, E., Li, X., Fan, D.P., Song, K., Liang, D., Lu, T., Luo, P., Shao, L.: Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 568–578 (2021)
31. Yu, H., Wu, J.: A unified pruning framework for vision transformers. Science China Information Sciences **66**(7), 1–2 (2023)
32. Zhai, X., Wang, X., Mustafa, B., Steiner, A., Keysers, D., Kolesnikov, A., Beyer, L.: Lit: Zero-shot transfer with locked-image text tuning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 18123–18133 (2022)